

GitHub Copilot Chat 速查表

符號、指令與語法指南

本指南整理了 GitHub Copilot Chat 的核心互動語法。
適用於 VS Code, Visual Studio, JetBrains, Xcode 以及 GitHub Web。

核心語法體系 (The Core Syntax System)



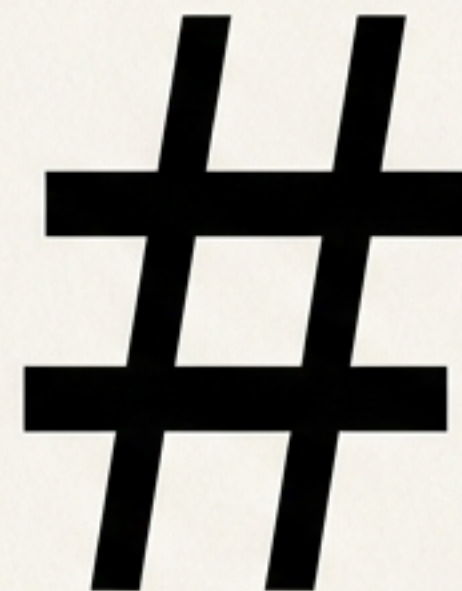
參與者 (Participants)

設定對話對象與領域專家。決定 “Who” 來回答。



斜線指令 (Slash Commands)

執行特定動作與任務。決定 “What” 要做什麼。



上下文變數 (Variables)

引用具體資料與文件。決定 “Which” 資料被使用。

符號 @：定義領域專家



定義 (Definition):

使用 @ 可以將對話鎖定在特定的專業領域，避免 AI 產生過於泛泛的回答。

語法結構：

@ + Agent Name (代理人名稱)

功能 (Function):

- 呼叫具備特定知識庫的參與者 (Participants)。
- 為對話附加相關的上下文環境 (Context)。

IDE 專用參與者 (IDE Specific Participants)

@workspace 擁有整個專案結構的上下文。

適用情境：詢問專案架構、不同代碼間的互動關係、設計模式。

@vscode 熟悉 VS Code 的指令與功能。

適用情境：詢問編輯器設定、除錯功能操作。

@terminal 熟悉終端機 Shell 與指令。

適用情境：協助編寫 Shell script、解釋終端機錯誤訊息。



雲端與平台參與者 (Cloud & Platform Participants)

@github (Web Context)

允許使用 GitHub 專屬技能。在 GitHub.com 對話框輸入 @ 可直接引用：

- Issues (議題)
- Pull Requests (拉取請求)
- Repositories (儲存庫)

@azure

擁有 Azure 服務的部署與管理知識 (Public Preview)。

適用情境：詢問 Azure 服務架構、部署流程。

符號 / ：執行斜線指令



定義 (Definition)：

使用 / 能夠讓 Copilot 執行預定義的複雜任務，無需編寫冗長的提示詞 (Prompts)。

語法結構: / + Command (指令)

功能 (Function)：

- 快速調用常見的開發任務（如解釋、修復、測試）。
- 管理對話 Session（如清除、新建）。

通用開發指令 (Universal Development Commands)

指令 (Command)	描述 (Description)
/explain	解釋目前編輯器中的程式碼運作原理 (Explain code functionality)
/fix	針對選取的程式碼提出錯誤修復建議 (Propose fixes)
/tests	為選取的程式碼生成單元測試 (Generate unit tests)
/help	顯示 Copilot 的使用指南與參考資訊 (Quick reference)

環境專屬指令 (Environment-Specific Commands)

Visual Studio

- `/optimize`：分析並改善程式碼的執行時間
- `/doc`：為符號加入文件註釋

Xcode

- `/simplify`：簡化目前的程式碼選取範圍

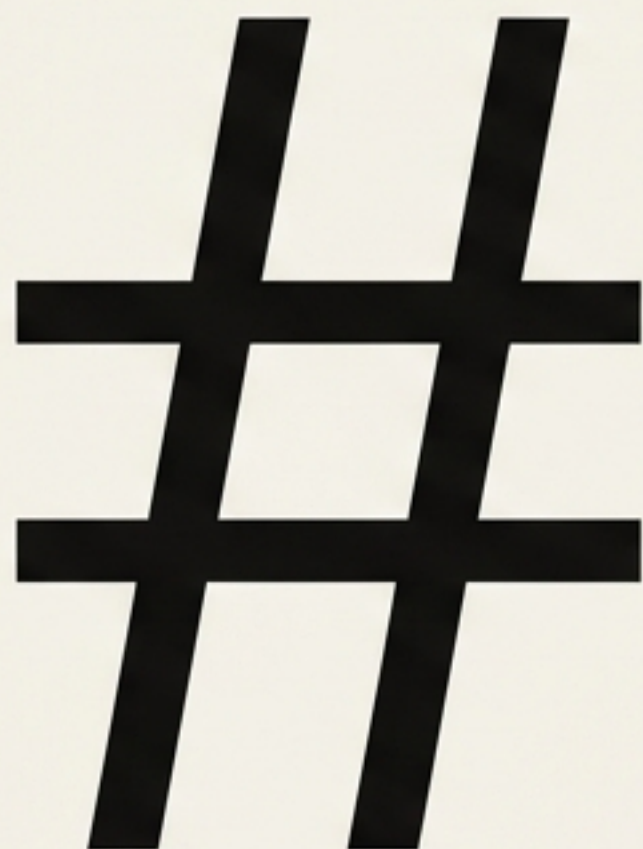
VS Code

- `/fixTestFailure`：尋找並修復失敗的測試

Session 管理 (通用)

- `/clear` (清除對話)
- `/new` (新對話)

符號 #：引用上下文變數



定義 (Definition)：

使用 # 明確指出 AI 應該參考哪些具體資料，解決“幻覺”問題並提升準確度。

語法結構： # + Variable (變數名稱)

功能 (Function)：

- 將特定的檔案、行數、或程式碼區塊注入到 Prompt 中。
- 不需要複製貼上代碼，直接引用即可。

檔案與編輯器變數 (File & Editor Variables)

#file

引用指定檔案的內容 (Includes specific file content)。

用法範例：解釋 #Main.java 的邏輯

#selection

引用目前游標選取的高亮文字 (Includes currently selected text)。

#editor

引用目前編輯器視窗的可見內容 (Active editor context)。

#solution

(Visual Studio) 引用當前的 Solution 上下文。

代碼結構變數 (Code Structure Variables)

#function -

引用目前的函式或方法

#class -

引用目前的類別

#line -

引用目前的程式碼行

#symbol -

引用目前的符號

路徑引用 (VS Code)

直接引用特定檔案的特定行數範圍。

#Main.cs: 66-72

網頁版進階技能：MCP (Web Advanced Skills: MCP)

在 GitHub 網頁版 Chat 中，Copilot 具備 MCP (Model Context Protocol) 技能，可直接執行操作。

無需特定符號，使用自然語言即可觸發。

create_branch

建立新分支 (Create a new branch)

search_users

搜尋使用者 (Search for users)

update_pull_request_branch

更新 PR 分支

push_files

推送檔案變更

組合技與最佳實踐 (Combinations)

解釋特定範圍

`[ /explain ] → [ #selection ]`

(解釋我選取的這段程式碼)

跨檔案查詢

`[ @workspace ] → [ #file:app.ts ] → [ #file:db.ts ]`

(詢問 workspace 代理人關於這兩個檔案的互動)

針對測試修復

`[ /fixTestFailure ] → [ #file:test_login.py ]`

(修復指定測試檔案中的失敗項目)

語法總表 (Syntax Matrix)

符號 (Symbol)	意義 (Meaning)	功能 (Function)	範例 (Example)
@	Who	設定領域專家	@workspace (問專案)
/	Action	執行指令動作	/tests (寫測試)
#	Data	引用具體資料	#file (看檔案)

透過語法提升效率 (Efficiency Through Syntax)

精準使用 @、/ 與 #，將 Copilot 從「聊天機器人」轉變為精確的「開發輔助工具」。

REFERENCE: GITHUB COPILOT CHAT CHEAT SHEET